

Prodapt — Capstone Project Specification

Multi-Framework Agentic AI · LangGraph · LlamaIndex · Google ADK · CrewAI · Streamlit

What I have provided for you

This folder contains **only starter assets**: policy documents (TXT), SQL scripts, and this specification HTML. **You must implement everything else** — Python modules, agents, orchestration, and the Streamlit UI — following the instructions below. Refer to your course labs for framework patterns.

[Business Use Case Overview](#) [Provided Assets](#) [Your Project Structure](#) [Database Rules](#) [What to Build](#) [Setup Steps](#) [Architecture](#) [Scenarios & Diagrams](#) [Evaluation](#)

1. Business Use Case — Why Agentic AI?

1.1 The Company

Prodapt is a mid-size telecom provider offering mobile voice, 4G LTE, and 5G data services across the United States. It serves consumer, business, and enterprise customers. Like every telecom operator, Prodapt receives thousands of inquiries daily about network problems, billing disputes, roaming, device upgrades, and service-level agreements (SLAs).

Today, these inquiries are handled by separate teams using separate tools. Your capstone project automates this fragmented workflow using a **multi-agent AI Operations Center**.

1.2 Who Uses the System?

Customer Support Agent (primary user)

Front-line staff who receive customer calls and chats. They need fast, accurate answers and often must coordinate between policy documents, network data, and billing systems — usually while the customer waits on hold.

Network Operations (NOC) staff

Engineers who monitor tower status, diagnose connectivity issues, and manage open incidents. Their expertise is technical; customer-facing explanations require a different tone.

Billing Resolution team

Specialists who investigate duplicate charges, apply credits, and follow dispute policy. Actions must comply with credit limits and leave an audit trail in the billing database.

End customer (indirect beneficiary)

Subscribers who expect one answer — not three transfers — when they ask: *"My 5G keeps dropping and I was charged twice — am I owed a credit?"*

1.3 The Existing Problem (As-Is — Manual Operations)

Before automation, a single customer inquiry can touch **four disconnected systems** and **three different teams**. There is no intelligent routing — humans decide where to look.

Today's pain points

- **Fragmented knowledge** — Policies live in PDF/TXT documents; outage and tower data live in SQL databases; live diagnostics sit in NOC dashboards; billing data sits in a separate CRM/billing system.
- **Slow resolution** — An agent puts the customer on hold, searches policy folders, emails NOC, waits for a billing supervisor, then manually drafts a reply. Average handle time: 15–25 minutes for complex cases.
- **Inconsistent answers** — Two agents may interpret the roaming policy or SLA credit rules differently because answers are not grounded in the same source documents.
- **Wrong-team routing** — Billing agents cannot diagnose tower issues; NOC engineers should not apply credits. Customers get transferred multiple times.
- **Multi-domain questions fail** — Questions like *"We had a 6-hour Midwest outage — am I eligible for SLA credit?"* require both operational data *and* policy interpretation. No single tool handles both.
- **High training cost** — New hires need weeks to learn which system to use for which question type.
- **No audit trail** — It is hard to prove which data sources and policies were consulted when a credit was applied or a diagnosis was given.

What the business needs

- One entry point where staff (or customers) ask a question in plain English
- Automatic routing to the right specialist — policy, analytics, network, or billing
- Answers grounded in actual policy documents and live database records
- Actionable outcomes — diagnostics run, credits applied — with data persisted in SQL
- Professional, consistent customer-ready responses every time
- Visibility into which agents ran and what they found (execution trace)
- Ability to scale specialist services independently (network vs billing microservices)

Example: How a billing dispute works today (manual)

flowchart TD C[Customer calls: charged twice for Unlimited Plus] --> A1[Support Agent] A1 -->|puts customer on hold| B1[Search billing CRM manually] B1 --> A1 A1 -->|transfer| B2[Billing Team] B2 -->|checks charges| B3[Spreadsheet / CRM lookup] B3 --> B2 B2 -->|reads policy PDF| B4[Find credit limit rules] B4 --> B2 B2 -->|applies credit manually| B5[Update billing system] B5 --> A1 A1 -->|writes email from scratch| C2[Customer receives reply 20+ min later]

Figure A — As-Is: manual, multi-team, no intelligent orchestration.

1.4 The Expected Solution (To-Be — Agentic AI Operations Center)

You will build an **AI Operations Center** that acts as an intelligent front door. A **LangGraph supervisor** understands the intent of each inquiry and delegates to specialist workers — each powered by the right framework for that job. The result is a single, coherent, customer-ready response.

Business Need	Your Solution Component	What It Automates
Answer policy & FAQ questions instantly	LlamaIndex Document RAG	Replaces manual PDF/TXT search. Retrieves relevant policy chunks and synthesizes

Business Need	Your Solution Component	What It Automates
		an answer from sla_policy.txt, roaming_policy.txt, etc.
Query operational data without writing SQL	LlamaIndex Semantic SQL	Replaces BI analyst or manual SQL. Automatically finds the right table (outages, towers, performance) and answers analytics questions.
Diagnose live network / tower issues	Google ADK Network Diagnostics (A2A)	Replaces NOC ticket lookup. Agent tools query tower status, performance, and open incidents from SQL and return diagnostic recommendations.
Investigate billing disputes and apply credits	Google ADK Billing Resolution (A2A)	Replaces billing team manual lookup. Agent tools read charges from SQL, detect duplicates, insert credits, update account balance.
Decide which specialist handles each step	LangGraph Supervisor	Replaces human judgment about routing. LLM supervisor reads conversation and picks the next worker or FINISH.
Deliver professional customer communication	CrewAI Communications Crew	Replaces agent manually drafting emails. Specialist drafts + reviewer approves tone, accuracy, and compliance.
Single interface for staff	Streamlit UI	Replaces switching between CRM, NOC dashboard, policy

Business Need	Your Solution Component	What It Automates
		folder, and email client.

Example: Same billing dispute with your agentic solution

flowchart TD
 C[Staff enters: CUST-10002 charged twice] --> UI[Streamlit UI]
 UI --> SUP[LangGraph Supervisor]
 SUP -->|billing intent| BR[BillingResolutionADK]
 BR -->|A2A HTTP| ADK[Billing ADK Agent port 8002]
 ADK -->|SQL| DB[(telecom_ops.db)]
 DB --> ADK
 ADK --> BR
 BR --> SUP
 SUP --> CREW[CrewAI Customer Comms]
 CREW --> SUP
 SUP --> UI
 UI --> OUT[Credit applied + professional customer letter + execution trace]

Figure B — To-Be: one query, automated routing, SQL-backed actions, polished response.

1.5 Problems This Use Case Solves

Business Problem	How Your Agentic System Solves It	Measurable Benefit
Knowledge silos across documents, SQL, NOC, and billing	Supervisor orchestrates LlamaIndex RAG, semantic SQL, and ADK agents from one entry point	One query instead of four systems
Slow average handle time on complex cases	Specialists run in sequence automatically; no hold time between teams	Minutes instead of 15–25 minutes
Inconsistent policy interpretation	RAG answers grounded in provided policy TXT files; billing follows billing_disputes_policy.txt rules	Same source documents every time
Wrong team / wrong tool selection	Supervisor routing matrix maps intent to PolicyRAG, NetworkAnalytics,	Correct specialist on first routing decision

Business Problem	How Your Agentic System Solves It	Measurable Benefit
	NetworkDiagnosticsADK, or BillingResolutionADK	
Multi-domain questions (outage + SLA, billing + policy)	Supervisor chains multiple workers; CrewAI merges all context into one response	Scenario 5 — combined flow handled end-to-end
Long onboarding for new support staff	System knows which framework and data source to use; staff only need to ask the question	Reduced training dependency on tribal knowledge
No audit trail for decisions	Streamlit execution trace shows every worker, ADK tool call outcome, and final CrewAI response	Transparent, reviewable agent steps
Monolithic agent hard to maintain	ADK specialists exposed via A2A as independent microservices; LangGraph consumes them remotely	Network and billing teams can deploy/update agents independently
Technical answers sent directly to customers	CrewAI crew always polishes the final response for tone, empathy, and compliance	Professional customer-ready communication

Business value summary

Prodapt's AI Operations Center does not replace humans entirely — it **automates the coordination, retrieval, and first-line resolution** that today consumes most of an agent's time. Humans remain in the loop for approvals (e.g., credits over \$50 per policy), escalations, and quality review — but the agentic system eliminates the friction of finding the right information in the right system at the right time.

1.6 Real Inquiry Types You Must Support

Your implementation must handle the same categories Prodapt staff see every day:

Inquiry Category	Real-World Example	Primary Automation
Roaming & international travel	"I'm traveling to Japan — what will data cost?"	PolicyRAG → CrewAI
SLA & outage credits	"Outage lasted 6 hours — do I get a credit?"	NetworkAnalytics + PolicyRAG → CrewAI
5G troubleshooting	"5G drops near tower TX-512 in Austin"	NetworkDiagnosticsADK → CrewAI
Billing disputes	"CUST-10002 charged twice for Unlimited Plus"	BillingResolutionADK → CrewAI
Network performance analytics	"Which towers have the highest packet loss?"	NetworkAnalytics → CrewAI
Device upgrade eligibility	"Can I trade in my iPhone 13?"	PolicyRAG → CrewAI

2. Project Overview

Build **Prodapt AI Operations Center** — an intelligent assistant for a fictional telecom provider. Customers and staff submit inquiries through a **Streamlit** UI. A **LangGraph supervisor** routes each request to the correct specialist worker.

2.1 What You Must Demonstrate

Capability	Framework	Example Question
Policy & FAQ answers	LlamaIndex RAG	What is the roaming policy for Europe?
Network analytics from SQL	LlamaIndex Semantic SQL	Which region had the most CRITICAL outages?
Live network diagnostics	Google ADK + A2A	My 5G drops near tower

Capability	Framework	Example Question
		TX-512 in Austin
Billing dispute resolution	Google ADK + A2A	Customer CUST-10002 was charged twice
Customer-ready responses	CrewAI	Final step for all customer-facing flows
Multi-agent routing	LangGraph	Supervisor decides which worker runs next
User interface	Streamlit	Query input, results, agent execution trace

Critical rules

- **LlamaIndex:** Use only for RAG and semantic SQL retrieval. Do **not** use LlamaIndex agents.
- **No in-memory dictionaries:** All operational data (towers, outages, billing, incidents) must be read from and written to the **SQLite database** created from the provided SQL scripts. ADK agent tools must execute SQL queries — never hard-code Python dicts for business data.
- **ADK agents** run as separate A2A HTTP services; LangGraph consumes them via RemoteA2aAgent.

2.2 Learning Objectives

1. Build document RAG with LlamaIndex VectorStoreIndex over the provided TXT policy files
2. Implement semantic SQL table retrieval using LlamaIndex ObjectIndex and SQLTableRetrieverQueryEngine
3. Expose Google ADK agents over the A2A protocol on separate ports
4. Consume remote ADK agents from LangGraph nodes using RemoteA2aAgent
5. Integrate a CrewAI sequential crew as a LangGraph worker node
6. Design a LangGraph supervisor with conditional routing and shared state
7. Deliver a Streamlit UI with agent execution trace
8. Implement all ADK tools against SQL tables — no in-memory data stores

3. Provided Starter Assets

I have provided you with the following files only. Do not modify the policy documents unless an extension task asks you to add new ones.

3.1 Policy Documents (RAG source)

Location: **data/documents/**

File	Contents	Typical Questions
sla_policy.txt	Uptime guarantees, outage credits, MTTR targets	Am I eligible for SLA credit after a 6-hour outage?
roaming_policy.txt	Zones A–D, Travel Pass, spend caps	What does roaming cost in Japan?
billing_disputes_policy.txt	Dispute window, credit limits, escalation	How long do I have to dispute a charge?
network_outage_procedures.txt	Incident classification, NOC procedures	What defines a CRITICAL outage?
5g_faq.txt	5G coverage, troubleshooting, device compatibility	Why is my 5G slow indoors?
device_upgrade_policy.txt	Trade-in values, early upgrade, installments	Can I upgrade my iPhone 13?

3.2 SQL Scripts

Location: **sql/**

File	Purpose	When to Run
01_schema.sql	Creates all tables (network, billing, incidents, performance)	First — before any seed data
02_seed_data.sql	Inserts sample towers, outages, subscriptions, billing charges,	Second — after schema

File	Purpose	When to Run
	open incidents	

After running both scripts, you will have a SQLite database (recommended path:

data/telecom_ops.db). All LlamaIndex semantic SQL queries and all ADK agent tools must use this database.

4. Project Directory Structure (You Create This)

Create a new project folder (or use the capstone folder and add your implementation alongside the provided assets). The tree below shows what is **provided** vs what **you must create**.

- [capstone-project/](#)
 - |— [capstone-project-specification.html](#) (provided)
 - |— [data/documents/*.txt](#) (6 policy files — provided)
 - |— [sql/01_schema.sql](#) (provided)
 - |— [sql/02_seed_data.sql](#) (provided)
 - |— [data/telecom_ops.db](#) (you create by running SQL scripts)
 - |— [data/vector_index/](#) (you create — persisted LlamaIndex embeddings)
 - |— [requirements.txt](#)
 - |— [.env](#) (API keys — never commit to git)
 - |— [llamaindex_rag/](#)
 - | |— [document_rag.py](#) — Vector RAG over policy TXT files
 - | |— [sql_semantic_search.py](#) — ObjectIndex + SQL query engine
 - |— [adk-services/](#)
 - | |— [network_diagnostics/](#)
 - | | |— [agent.py](#) — ADK agent, A2A port 8001, SQL-backed tools
 - | |— [billing_resolution/](#)
 - | | |— [agent.py](#) — ADK agent, A2A port 8002, SQL-backed tools
 - |— [orchestration/](#)
 - | |— [state.py](#) — LangGraph AgentState definition

- | | — `graph.py` — Supervisor + all worker nodes + graph compile
- | | — `adk_remote_client.py` — RemoteA2aAgent wrapper for LangGraph
- | | — `crew_nodes.py` — CrewAI customer communications crew
- | — `ui/`
- | — `app.py` — Streamlit application

5. Database Design & Usage Rules

No in-memory dictionaries

Do **not** store tower status, billing accounts, charges, or incidents in Python dictionaries, lists, or global variables. Every ADK tool must **query or update SQLite**. This mirrors real production systems where agents interact with persistent data stores.

5.1 Tables Provided in 01_schema.sql

Table	Used By	Purpose
network_towers	LlamaIndex SQL, Network Diagnostics ADK	Tower inventory: region, city, technology, status
network_outages	LlamaIndex SQL	Historical outage records with severity and root cause
tower_performance	LlamaIndex SQL, Network Diagnostics ADK	Latency, packet loss, throughput, signal strength
open_incidents	Network Diagnostics ADK	Active NOC incidents linked to towers
customer_subscriptions	LlamaIndex SQL	Plan names, fees, account types by region
billing_accounts	Billing Resolution ADK	Current balance per customer
billing_charges	Billing Resolution ADK	Line items on bills; duplicate flag column for disputes
billing_credits	Billing Resolution ADK	Applied or pending credits

Table	Used By	Purpose
		(insert on credit approval)
billing_disputes	Billing Resolution ADK	Open and resolved dispute records

5.2 How Each Component Uses the Database

Component	Database Interaction
LlamaIndex Semantic SQL	Connect via SQLiteDatabase wrapper. ObjectIndex selects relevant tables; query engine generates and runs SQL. Read-only for analytics scenarios.
Network Diagnostics ADK tools	check_tower_status — JOIN network_towers with latest tower_performance and open_incidents. run_connectivity_diagnostics — Read tower status and performance; return recommendations based on SQL results. get_regional_network_summary — Aggregate tower counts and statuses by region from SQL.
Billing Resolution ADK tools	lookup_billing_account — SELECT from billing_accounts JOIN billing_charges. check_duplicate_charges — SELECT charges WHERE is_duplicate_flag = 1 or same description/period. apply_billing_credit — INSERT into billing_credits, UPDATE billing_accounts balance. Credits over policy limit go to PENDING_APPROVAL status in SQL.

5.3 Semantic SQL Table Context Strings

When building the LlamaIndex ObjectIndex, write a human-readable context description for each table so the embedding model can semantically match user questions to the right table.

Example intent (you write the actual strings in your implementation):

- **network_outages** — describe outage severity, affected customers, incident IDs
 - **network_towers** — describe tower locations, technology, operational status
 - **tower_performance** — describe latency, packet loss, throughput metrics
 - **customer_subscriptions** — describe plans, fees, account types
-

6. Detailed Build Instructions (No Code — Design Only)

6.1 LlamaIndex Document RAG (document_rag.py)

1. Configure a local embedding model (HuggingFace all-MiniLM-L6-v2 is recommended from your lab).
2. Load all TXT files from data/documents/ using SimpleDirectoryReader.
3. Build a VectorStoreIndex and persist embeddings to data/vector_index/ so rebuilds are not required every run.
4. Expose a single function that accepts a natural language question and returns a string answer.
5. Use as_query_engine — not LlamaIndex agents, workflows, or ReAct patterns.

6.2 LlamaIndex Semantic SQL (sql_semantic_search.py)

1. Connect to data/telecom_ops.db using the LlamaIndex SQLiteDatabase wrapper.
2. Create SQLTableNodeMapping for all tables in the schema.
3. Define SQLTableSchema objects with context strings for semantic table selection.
4. Build ObjectIndex over table schemas using VectorStoreIndex.
5. Create SQLTableRetrieverQueryEngine with similarity_top_k of 1 or 2.
6. Expose a single function that accepts a question and returns the synthesized answer.

6.3 Network Diagnostics ADK Service (port 8001)

1. Create a Google ADK Agent with a clear instruction describing NOC diagnostics responsibilities.
2. Implement three async tools — each tool opens a SQLite connection and runs SQL:
 - `check_tower_status(tower_id)` — returns status, technology, signal metrics, any open incident
 - `run_connectivity_diagnostics(tower_id, symptom)` — reads tower data from SQL, returns recommendations
 - `get_regional_network_summary(region)` — aggregates tower health for a region from SQL
3. Wrap the agent with `to_a2a` and run with `uvicorn` on port 8001.
4. Verify the agent card is published at `localhost:8001/.well-known/agent-card.json`

6.4 Billing Resolution ADK Service (port 8002)

1. Create a Google ADK Agent for billing dispute resolution.
2. Implement three async tools — all SQL-backed:
 - `lookup_billing_account(customer_id)` — balance and recent charges from SQL
 - `check_duplicate_charges(customer_id)` — find flagged duplicates in `billing_charges`
 - `apply_billing_credit(customer_id, amount, reason)` — INSERT credit row, UPDATE account balance; use `PENDING_APPROVAL` in SQL when amount exceeds policy auto-approval limit (\$50 per `billing_disputes_policy.txt`)
3. Expose via `to_a2a` on port 8002.

6.5 LangGraph Remote ADK Client (`adk_remote_client.py`)

1. Create `RemoteA2aAgent` instances pointing to each agent card URL.
2. Wrap each remote agent in a lightweight proxy ADK Agent with the remote as `sub_agent`.
3. Use `ADK Runner.run_async` to send messages and collect final responses.
4. Expose synchronous wrapper functions for use inside LangGraph nodes (Network Diagnostics and Billing Resolution).
5. Handle service-unavailable errors gracefully with a clear message if ports 8001/8002 are not running.

6.6 CrewAI Customer Communications (`crew_nodes.py`)

1. Create a two-agent sequential crew:
 - Communications Specialist — drafts response using original query + accumulated

agent_context from prior workers

- Quality Reviewer — reviews for accuracy, tone, and policy compliance; outputs final text only
2. Use Process.sequential. Reviewer task depends on drafter task output.
 3. Expose one function: accepts user query and context string, returns final customer-ready response.

6.7 LangGraph Orchestration (graph.py + state.py)

State (AgentState) must include:

- messages — conversation history (append-only across nodes)
- next — supervisor routing decision
- user_query — original question
- agent_context — accumulated outputs from all workers (passed to CrewAI)

Worker nodes to implement:

Node Name	Action	Returns To
supervisor	LLM with structured output picks next worker or FINISH	Conditional edge to worker or END
policy_rag	Calls LlamaIndex document RAG	supervisor
network_analytics	Calls LlamaIndex semantic SQL	supervisor
network_diagnostics_adk	Calls remote ADK via adk_remote_client	supervisor
billing_resolution_adk	Calls remote ADK via adk_remote_client	supervisor
customer_comms_crew	Calls CrewAI crew with full agent_context	supervisor

Every worker node updates agent_context with its output before returning to the supervisor. The supervisor routes to CustomerCommsCrew last (before FINISH) for all customer-facing flows.

6.8 Streamlit UI (ui/app.py)

The Streamlit app is the single entry point for staff and demo evaluators. It must make the multi-agent orchestration **visible** — especially which workers ran and what each specialist returned before CrewAI polished the final answer.

6.8.1 Page layout

Area	Required elements
Main content	Page title (e.g. <i>Prodapt AI Operations Center</i>); brief caption naming the frameworks (LangGraph, LlamaIndex, Google ADK, CrewAI); customer inquiry text area; primary Submit button; final customer response; Agent Execution Trace panel (see 6.8.3)
Sidebar	System status indicators; framework map (which capability uses which framework). Do not require sample-query buttons or preset query shortcuts in the sidebar — students type or paste inquiries in the main text area for demos.

6.8.2 Sidebar — system status & framework map

The sidebar must help users know whether the system is ready before submitting a query:

- **Database status** — indicate whether data/telecom_ops.db exists (e.g. Ready / Run init_db.py)
- **Vector index status** — indicate whether data/vector_index/ is built, or note that first RAG query will build it
- **ADK service status** — live check (HTTP) for Network Diagnostics on port **8001** and Billing Resolution on port **8002**; show Running or Not running
- **ADK reminder** — if either ADK service is down, show a short warning with the commands to start both agent.py services
- **Framework map** — a compact table or list mapping capability → framework (Policy FAQ → LlamaIndex RAG; analytics → Semantic SQL; diagnostics/billing → ADK A2A; orchestration →

LangGraph; final comms → CrewAI)

6.8.3 Main area — inquiry, response & Agent Execution Trace

Required — Agent Execution Trace

This is a **mandatory** demo and evaluation feature. Evaluators use it to verify correct routing (PolicyRAG vs NetworkAnalytics vs ADK workers vs CustomerCommsCrew) without reading server logs.

1. **Customer inquiry input** — multi-line text area; placeholder text inviting plain-English questions; bound to session state if you use programmatic updates elsewhere
2. **Submit button** — invokes your graph entry function (e.g. `run_telecom_assistant(user_query)`); show a spinner while the graph runs
3. **Final customer response** — display `final_response` prominently (e.g. success/info box) after the run completes; this is the CrewAI-polished text shown to the customer
4. **Agent Execution Trace** — an expander (recommended: **expanded by default** on results) that lists every worker executed in order. For each step show:
 - **Step number** (1, 2, 3, ...)
 - **Worker name** exactly as in LangGraph (PolicyRAG, NetworkAnalytics, NetworkDiagnosticsADK, BillingResolutionADK, CustomerCommsCrew)
 - **Truncated worker output** — the text that worker contributed to `agent_context` (truncate long outputs for readability, e.g. first 500 characters)
5. **Empty trace handling** — if no workers ran, show a clear message inside the trace panel
6. **Error handling** — if the graph throws, show `st.error` with the message; do not leave the UI in an ambiguous state

6.8.4 Data contract (LangGraph → Streamlit)

Your graph entry function should return a dict the UI can render without parsing messages manually:

Key	Type	UI use
<code>final_response</code>	<code>str</code>	Main customer-facing answer

Key	Type	UI use
		(from CustomerCommsCrew)
execution_trace	list of dicts	Agent Execution Trace panel; each item: {"worker": " <name>", "output": " <truncated text>"}
agent_context	str (optional)	May be shown in a secondary debug expander for me to review; not required for minimum spec

Populate execution_trace inside each LangGraph worker node when the worker completes — append one entry per worker invocation, in execution order.

6.8.5 Example trace (Scenario 4 — billing dispute)

Expected trace appearance

Customer CUST-10002 was charged twice for Unlimited Plus. Investigate and apply credit.

Step 1 — BillingResolutionADK → Step 2 — CustomerCommsCrew

Step 1 output summarizes SQL tool results (duplicate charge found, credit applied or pending).

Step 2 output is the polished letter; the main response area shows the final Step 2 text.

6.8.6 Optional enhancements (not required for full marks)

- Secondary expander: raw agent_context for debugging
- Sidebar sample-query buttons for local testing (see implementation guide — recommended for development, not part of this specification)
- Clear button or query history

7. Setup & Run Order

Create venv & install packages

langgraph, langchain-openai, llama-index, google-adk[a2a], crewai, streamlit, python-dotenv, sentence-transformers

Configure .env

Set OPENAI_API_KEY. Optionally set ADK model and agent card URLs.

Build database

Run 01_schema.sql then 02_seed_data.sql against data/telecom_ops.db using sqlite3 CLI or DB tool.

Implement & test modules

LlamaIndex RAG, semantic SQL, then ADK services, then LangGraph, then Streamlit.

Start ADK services

Terminal 1: network diagnostics port 8001. Terminal 2: billing resolution port 8002.

Start Streamlit

Terminal 3: streamlit run ui/app.py from project root.

7.1 Database Initialization (Manual)

Using the sqlite3 command-line tool (or any SQL client):

1. Ensure the data/ folder exists in your project.
2. Create or open data/telecom_ops.db.
3. Execute sql/01_schema.sql in full.

4. Execute sql/02_seed_data.sql in full.
5. Verify tables are populated — e.g. network_towers should have 10 rows; billing_charges should include a duplicate row for CUST-10002.

8. System Architecture Diagrams

8.1 High-Level Architecture

```

flowchart TB
    subgraph UI ["Streamlit UI"]
        U[Customer Query]
        T[Execution Trace]
    end
    subgraph LG ["LangGraph Supervisor"]
        S[Supervisor Router]
        PR[PolicyRAG]
        NA[NetworkAnalytics]
        ND[NetworkDiagnosticsADK]
        BR[BillingResolutionADK]
        CC[CustomerCommsCrew]
    end
    subgraph LI ["LlamaIndex Retrieval Only"]
        VR[VectorStoreIndex on TXT docs]
        OS[ObjectIndex Semantic SQL]
    end
    subgraph ADK ["Google ADK A2A Services"]
        NDS[Network Diagnostics port 8001]
        BRS[Billing Resolution port 8002]
    end
    subgraph DB ["SQLite telecom_ops.db"]
        SQL[(All Tables)]
    end
    U --> S
    S --> PR & NA & ND & BR & CC
    PR --> VR
    NA --> OS
    OS --> SQL
    ND --> |RemoteA2aAgent| NDS
    BR --> |RemoteA2aAgent| BRS
    NDS --> SQL
    BRS --> SQL
    CC --> T
  
```

Figure 1 — All business data lives in SQLite. No in-memory dictionaries.

8.2 LangGraph Supervisor Loop

```

flowchart LR
    START((START)) --> SUP[Supervisor]
    SUP --> |worker| W[Specialist Node]
    W --> SUP
    SUP --> |FINISH| END((END))
  
```

Figure 2 — Every worker returns to supervisor until FINISH.

8.3 ADK A2A Remote Pattern

```

flowchart LR
    LG[LangGraph Node] --> RC[Remote Client Module]
    RC --> |HTTP A2A| SVC[ADK Service Process]
    SVC --> AG[ADK Agent]
    AG --> T[SQL Tools]
    T --> DB[(telecom_ops.db)]
  
```

Figure 3 — ADK tools query SQLite; LangGraph never imports agent logic directly.

9. End-to-End Scenarios with Sequence Diagrams

9.1 Scenario 1 — Policy / FAQ (LlamaIndex Document RAG)

Sample query

What is Prodapt's roaming policy for Western Europe?

Supervisor → PolicyRAG → Supervisor → CustomerCommsCrew → FINISH

sequenceDiagram
autonumber actor User participant UI as Streamlit participant Sup as Supervisor participant RAG as PolicyRAG participant LI as LlamaIndex VectorIndex participant Docs as Policy TXT Files participant Crew as CrewAI
User->>UI: Roaming policy for Europe?
UI->>Sup: invoke graph
Sup->>RAG: route
RAG->>LI: query policy documents
LI->>Docs: retrieve relevant chunks
Docs-->>LI: roaming_policy.txt content
LI-->>RAG: synthesized answer
RAG->>Sup: update agent_context
Sup->>Crew: route
Crew-->>Sup: polished response
Sup->>Sup: FINISH
Sup-->>UI: result + trace
UI-->>User: display

Figure 4 — Policy questions never touch SQL or ADK services.

9.2 Scenario 2 — Network Analytics (LlamaIndex Semantic SQL)

Sample query

Which region had the most CRITICAL network outages recently?

Supervisor → NetworkAnalytics → Supervisor → CustomerCommsCrew → FINISH

sequenceDiagram
autonumber actor User participant UI as Streamlit participant Sup as Supervisor participant NA as NetworkAnalytics participant OI as ObjectIndex participant QE as SQL Query Engine participant DB as telecom_ops.db participant Crew as CrewAI
User->>UI: Most CRITICAL outages by region?
UI->>Sup: invoke graph
Sup->>NA: route
NA->>OI: query
OI->>QE: query
QE->>DB: query
DB-->>QE: results
QE-->>OI: results
OI-->>NA: results
NA-->>Sup: results
Sup->>Crew: route
Crew-->>Sup: polished response
Sup->>Sup: FINISH
Sup-->>UI: result + trace
UI-->>User: display

NA->>OI: semantically match question to tables OI-->>NA: selects network_outages table
NA->>QE: generate and run SQL QE->>DB: SELECT region, COUNT WHERE
severity=CRITICAL DB-->>QE: Midwest has most CRITICAL outages QE-->>NA: natural
language answer NA->>Sup: update agent_context Sup->>Crew: route CustomerCommsCrew
Crew-->>Sup: formatted summary Sup-->>UI: FINISH

Figure 5 — ObjectIndex picks network_outages before SQL is generated.

9.3 Scenario 3 — Network Diagnostics (ADK A2A + SQL Tools)

Sample query

My 5G keeps dropping in Austin near tower TX-512. Please diagnose.

Supervisor → NetworkDiagnosticsADK → Supervisor → CustomerCommsCrew → FINISH

sequenceDiagram
autonumber actor User participant UI as Streamlit participant Sup as Supervisor participant ND as NetworkDiagnostics Node participant RC as RemoteA2a Client participant A2A as ADK Service port 8001 participant AG as Network Diagnostics Agent participant DB as telecom_ops.db participant Crew as CrewAI
User->>UI: 5G drops near TX-512
UI->>Sup: invoke graph
Sup->>ND: route NetworkDiagnosticsADK
ND->>RC: invoke remote agent
RC->>A2A: A2A HTTP request
A2A->>AG: process request
AG->>DB: SQL check_tower_status TX-512
DB-->>AG: OPERATIONAL 5G signal -72 dBm
AG->>DB: SQL run_connectivity_diagnostics
DB-->>AG: performance metrics
AG-->>A2A: diagnostic report
A2A-->>RC: response
RC-->>ND: result text
ND->>Sup: update agent_context
Sup->>Crew: route CustomerCommsCrew
Crew-->>UI: empathetic customer response

Figure 6 — ADK tools read tower and incident data from SQL, not Python dicts.

9.4 Scenario 4 — Billing Dispute (ADK A2A + SQL Tools)

Sample query

Customer CUST-10002 was charged twice for Unlimited Plus. Investigate and apply credit.

Supervisor → BillingResolutionADK → Supervisor → CustomerCommsCrew → FINISH

sequenceDiagram
 actor User
 participant UI as Streamlit
 participant Sup as Supervisor
 participant BR as BillingResolution
 participant RC as RemoteA2a Client
 participant A2A as ADK Service port 8002
 participant AG as Billing Resolution Agent
 participant DB as telecom_ops.db
 participant Crew as CrewAI
 User->>UI: CUST-10002 charged twice
 UI->>Sup: invoke graph
 Sup->>BR: route BillingResolutionADK
 BR->>RC: invoke remote agent
 RC->>A2A: A2A request
 A2A->>AG: process dispute
 AG->>DB: SELECT billing_accounts and
 billing_charges
 DB-->>AG: balance 131.98 two 65.99 charges one duplicate
 AG->>DB: INSERT billing_credits
 UPDATE billing_accounts
 DB-->>AG: credit applied new balance 65.99
 AG-->>A2A: resolution summary
 A2A-->>BR: response
 BR->>Sup: update agent_context
 Sup->>Crew: route CustomerCommsCrew
 Crew-->>UI: professional billing letter

Figure 7 — Credits and balance updates persist in SQLite via ADK tools.

9.5 Scenario 5 — Combined Multi-Worker Flow

Sample query

We had a 6-hour outage in the Midwest. Am I eligible for an SLA credit and what does policy say?

Supervisor → NetworkAnalytics → Supervisor → PolicyRAG → Supervisor →
 CustomerCommsCrew → FINISH

sequenceDiagram
 actor User
 participant Sup as Supervisor
 participant NA as NetworkAnalytics
 participant DB as telecom_ops.db
 participant PR as PolicyRAG
 participant Docs as sla_policy.txt
 participant Crew as CrewAI
 User->>Sup: 6-hour Midwest outage SLA credit?
 Sup->>NA: need outage facts
 NA->>DB: semantic SQL on network_outages
 DB-->>NA: CRITICAL 6 hours 8500 customers
 NA->>Sup: agent_context updated
 Sup->>PR: need policy rules
 PR->>Docs: retrieve SLA credit eligibility
 Docs-->>PR: Business 4hr Consumer 8hr thresholds
 PR->>Sup: agent_context updated
 Sup->>Crew: synthesize combined answer
 Crew-->>Sup: eligibility determination with policy citation
 Sup->>Sup: FINISH

Figure 8 — Supervisor sequences multiple specialists; CrewAI merges all context.

9.6 Supervisor Routing Decision Matrix

Question Type	Signals / Keywords	First Worker	Then
Policy / FAQ	policy, roaming, SLA, 5G FAQ, upgrade, rules	PolicyRAG	CustomerCommsCrew → FINISH
Analytics	outage trends, packet loss, latency, how many, top N	NetworkAnalytics	CustomerCommsCrew → FINISH
Connectivity	signal drops, tower ID, diagnose, not working	NetworkDiagnosticsADK	CustomerCommsCrew → FINISH
Billing	charged twice, credit, dispute, CUST- prefix	BillingResolutionADK	CustomerCommsCrew → FINISH
Combined	outage + policy, billing + policy	Data or ADK worker first	Other worker → CustomerCommsCrew → FINISH

10. Evaluation Criteria & Demo

Criteria	Weight	What Evaluators Check
Correct framework per scenario	30%	Policy→RAG; analytics→semantic SQL; diagnostics/billing→ADK A2A; comms→CrewAI
LangGraph supervisor	20%	Structured routing, worker loop, agent_context accumulation, FINISH after CrewAI
LlamaIndex RAG + semantic SQL	20%	Vector index on docs; ObjectIndex table selection; no LlamaIndex agents

Criteria	Weight	What Evaluators Check
ADK A2A + SQL tools	20%	Two services running; RemoteA2aAgent from LangGraph; tools use SQL not dicts
Streamlit UX	10%	Clean UI; sidebar system status + framework map; prominent final response; Agent Execution Trace with step order, worker names, and truncated outputs

10.1 Fifteen-Minute Demo Script

1. Explain business use case (Section 1) and architecture (Section 8)
2. Show database populated from SQL scripts — point out billing duplicate row for CUST-10002
3. Start both ADK services; show agent cards in browser
4. Run Scenario 1 — policy question; show PolicyRAG in trace
5. Run Scenario 2 — outage analytics; explain semantic table selection
6. Run Scenario 3 — tower diagnostic; confirm SQL was queried (not hard-coded data)
7. Run Scenario 4 — billing dispute; show credit row inserted in billing_credits table
8. Open **Agent Execution Trace** — walk through each step (worker name + output); confirm routing matches scenario
9. Show CrewAI polished response vs specialist output in the trace

10.2 Optional Extensions

Extension	Framework
HITL approval for credits over \$50	Streamlit + LangGraph interrupt
LangSmith tracing of supervisor decisions	LangGraph
Add fiber_backhaul_policy.txt and rebuild vector index	LlamaIndex

Extension	Framework
Parallel PolicyRAG + NetworkAnalytics using LangGraph Send API	LangGraph
MCP tool on Network Diagnostics ADK agent	Google ADK

10.3 Troubleshooting

Problem	Fix
ADK service unavailable	Start both agent.py services on ports 8001 and 8002 before Streamlit
Empty SQL results	Re-run 01_schema.sql and 02_seed_data.sql
Wrong worker routed	Improve supervisor system prompt with clearer routing rules
Tools return stale/hard-coded data	Ensure tools query SQLite — remove any in-memory dicts
First RAG query slow	Normal — embeddings build once and persist in data/vector_index/

Start here

Run the SQL scripts → implement LlamaIndex modules → build ADK services with SQL tools → wire LangGraph → add CrewAI → build Streamlit. Use your course labs as reference for each framework. Good luck!